

Lekcja 12 - Lakehouse / Delta Lake / Iceberg

2026-09-03

Data Engineering Course

Od plików do tabeli z historią, ACID i rollbackiem.

1. Problem z plain Parquet
2. Czym jest lakehouse
3. Delta Lake: commit log + ACID
4. Time travel i rollback
5. Schema evolution
6. Delta vs Iceberg
7. Różnica między plikami a tabelą
8. Lab + checklist

■ Problem biznesowy

Masz pipeline, który zapisuje dane do katalogu z Parquetem.

```
raw/orders/  
  part-0001.parquet  
  part-0002.parquet  
  part-0003.parquet
```

To działa, dopóki:

- jest jeden writer,
- jest mało jobów,
- nie trzeba wrócić do poprzedniego stanu,
- nie zmienia się schema.

Rzeczywistość jest inna:

```
partial write  
concurrent writes  
bad overwrite  
brak historii  
brak audytu
```

■ Co psuje się w plain Parquet?

▣ 1. Partial write

Job zapisuje 10 partycji.
Po 7 padł proces.
Czytający widzi 7 plików, nie 10.

Dane są niekompletne. Każdy kolejny job widzi "częściowy stan".

▣ 2. Concurrent writes

Job A i Job B robią overwrite do tego samego katalogu.
Jedna operacja nadpisuje drugą.

▣ 3. Brak rollbacku

Bug zmienia 3M rekordów.
Nie ma poprzedniej wersji.
Dane są stracone.

■ Dlaczego to jest ważne?

```
Parquet = format pliku  
Delta = tabela + historia + transakcje
```

Dane w produkcji to nie tylko pliki. To stan systemu.

Jeśli tabela może być nadpisana przez kilka jobów, to potrzebujemy:

- atomowości,
- historii zapisów,
- walidacji schematu,
- możliwości powrotu do wersji sprzed błędu.

Co to jest tabela danych?

Tabela to nie folder z plikami. Tabela to:

- nazwa,
- schemat,
- stan danych,
- historia zmian,
- semantyka operacji na danych.

Parquet daje pliki. Delta daje tabelę z metadanymi i logiem transakcji.

Plain Parquet

- > katalog z plikami
- > brak semantyki tabeli
- > brak historii
- > trudny rollback

Lakehouse / Delta

- > tabela z commit logiem
- > ACID semantics
- > time travel
- > schema evolution

To nie jest kosmetyka. To zmienia sposób, w jaki działają pipeline'y.

■ Czym jest lakehouse?

Lakehouse łączy dwie rzeczy:

1. obejmowanie danych w data lake (s3 / ADLS / lokalny filesystem)
2. semantykę tabeli jak w warehouse (schema, transakcje, wersje)

Czyli:

- przechowujemy surowe dane w formacie open,
- ale zachowujemy kontrolę nad stanem i integracją tabeli.

Przykład:

bronze/	-> surowe dane
silver/	-> oczyszczone dane
gold/	-> agregacje biznesowe

Delta jest jednym z najważniejszych formatów lakehouse.

[illegible]

```
data/delta/orders/  
├── delta_log/  
│   ├── 000000000000000000000000000000000000.json  
│   ├── 000000000000000000000000000000000001.json  
│   └── 000000000000000000000000000000000002.json  
├── part-00000-abc.parquet  
├── part-00001-def.parquet  
└── part-00002-ghi.parquet
```

delta log

To jest klucz do zrozumienia:

- Dane są w Parquet,
- ale metadane o zmianach są w logu Delta.

■ Co robi `delta_log`?

Każdy zapis jest traktowany jako commit.

Delta zapisuje:

- wersję tabeli,
- operację,
- schemat,
- pozycję nowych plików,
- informacje o zmianach.

To pozwala na:

- atomic zapis,
- zgodność przy równoczesnym zapisie,
- odczyt starej wersji,
- monitorowanie historii operacji.

Atomicity	= cały zapis albo nic
Consistency	= stan musi być spójny
Isolation	= konflikty między jobami są wykrywane
Durability	= log commit jest trwały

Przykład:

Pipeline zapisuje 1000 rekordów.
Po 700 przestaje działać.
Delta nie zostawia częściowego stanu.

To jest różnica między "zapisem na pliki" a "stanem tabeli".

■ Dlaczego ACID ważne dla danych?

W raportach, warehouse i BI nie wolno mieć sytuacji:

- częściowo zapisanych danych,
- uszkodzonego stanu po overwrite,
- dat z różnym poziomem jakości,
- niejasnego pochodzenia błędu.

ACID nie jest akademickie. To jest narzędzie do:

- bezpieczeństwa danych,
- zrozumienia produkcji,
- utrzymania poprawności procesu ETL.

■ Time travel

Najważniejsza rzecz w Delta poza ACID: historia danych.

```
spark.read.format("delta") \  
  .option("versionAsOf", 0) \  
  .load("data/delta/orders")
```

Możesz odczytać:

- wcześniejszą wersję tabeli,
- stan z konkretnego czasu,
- wersję przed błędnym deployem.

```
spark.read.format("delta") \  
  .option("timestampAsOf", "2026-09-01 12:00:00") \  
  .load("data/delta/orders")
```

■ Time travel – kiedy to pomaga?

- błąd w pipeline wymazał 2 miliony rekordów
- deploy nadpisał dane wygenerowane w nocy
- trzeba sprawdzić, co było w ubiegły poniedziałek
- trzeba wykonać rollback bez rebuildu całego storage

To jest bardzo ważne w produkcji.

Business answer:

"Nie mamy już danych?"

"Tak mamy – w wersji 3, zanim błąd się pojawił."

Historia operacji

```
from delta.tables import DeltaTable

dt = DeltaTable.forPath(spark, "data/delta/orders")

dt.history().select("version", "timestamp", "operation").show()
```

`history()` pokazuje:

- która wersja była zapisana,
- kiedy,
- jaki typ operacji został wykonany,
- czy to był append, overwrite, merge, vacuum itd.

To jest audyt i podstawowy narzędziowy mechanizm dla platformy.

■ Schema evolution

W realnych danych schema się zmienia.

```
new_df = df.withColumn("region", F.lit("EU"))

new_df.write.format("delta") \
    .option("mergeSchema", "true") \
    .mode("append") \
    .save("data/delta/orders")
```

Co dzieje się po dodaniu kolumny?

- nowe rekordy mają region,
- stare rekordy mają NULL,
- tabela nie pęka od razu,
- można świadomie rozbudować schema.

Bez `mergeSchema` Spark zgłasza błąd.

■ Schema drift – real problem

```
Tydzień temu tabela miała: order_id, total_amount  
Dziś pojawia się: order_id, total_amount, region,  
status_reason
```

To nie jest błąd. To normalny drift danych.

Wielkie dane = zmieniają się kolumny, source'y, payloady i KAFKA.

W Delta można to kontrolować. W plain Parquet jest to często "manual repair".

Jak skonfigurować Spark do Delta?

```
from delta import configure_spark_with_delta_pip
from pyspark.sql import SparkSession

spark = (
    configure_spark_with_delta_pip(
        SparkSession.builder
            .appName("DeltaLakeDemo")
            .master("local[*]")
            .config("spark.sql.extensions", "
io.delta.sql.DeltaSparkSessionExtension")
            .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
        )
    .getOrCreate()
)
```

Ważne: nie chodzi tylko o `format("delta")`. Trzeba też dodać rozszerzenie Spark i katalog Delta.

Zapis i odczyt w praktyce

```
df.write.format("delta").mode("overwrite").save("data/delta/orders")

orders = spark.read.format("delta").load("data/delta/orders")
```

Zwykle `read` i `write` działają, ale w Delta:

- zapis ma log transakcyjny,
- odczyt może odwoływać się do wersji,
- historia jest dostępna na poziomie tabeli.

■ Delta vs Iceberg

Delta Lake

- > mocne w Spark / Databricks / Azure
- > silne wsparcie dla time travel + schema evolution

Apache Iceberg

- > bardziej neutralny format tabeli w wielu silnikach
- > mocny w ekosystemie AWS / Trino / Flink / Spark

■ Kiedy co wybrać?

Databricks / Azure / Spark-native	-> Delta Lake
AWS + Athena / Glue + multi-engine	-> Iceberg

Nie chodzi o to, który jest "najlepszy", tylko który jest naturalny dla platformy.

Parquet

- > szybki odczyt
- > prosty format pliku
- > brak historii
- > brak transakcji
- > łatwe overwrite bugi

Delta

- > Parquet + commit log + historia
- > ACID na plikach
- > snapshot history
- > schema evolution

Krótkie zdanie do zapamiętania:

Delta nie jest "lepszy Parquet" jako plik.
Delta jest lepszą tabelą na danych lakehouse.

Warehouse: tabela ma semantykę, transakcje i kontrolę
Lake: dane są elastyczne, tańsze, lecz mniej „tabelowe”
Lakehouse: łączy oba modele

Lakehouse daje:

- przechowywanie w open format,
- skalowalność Data Lake,
- kontrolę tabeli jak w warehouse.

To jest najważniejsza architektura w nowoczesnych platformach danych.

■ Jak to wygląda w pipeline?

```
bronze/*      -> surowe dane, bez wzorcowania  
silver/*     -> czyste dane, walidacja i  
normalizacja  
gold/*       -> agregacje biznesowe dla raportów
```

Delta wchodzi głównie w warstwy silver/gold, bo tam:

- trzeba mieć poprawne versioning,
- trzeba kontrolować schema,
- trzeba wracać po błędach,
- trzeba dawać spójne źródło dla raportów.

DELTA LAKE – stały wzorzec

```
=== SparkSession ===
from delta import configure_spark_with_delta_pip
from pyspark.sql import SparkSession

spark = configure_spark_with_delta_pip(
    SparkSession.builder
        .appName("DeltaDemo")
        .master("local[*]")
        .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        ).getOrCreate()

=== Zapis ===
df.write.format("delta").mode("overwrite").save("data/delta/orders")

=== Odczyt ===
spark.read.format("delta").load("data/delta/orders")

=== Time travel ===
spark.read.format("delta").option("versionAsOf",
```

Checklist przed labem

- [] Potrafię wyjaśnić, czym jest plain Parquet i gdzie ma limit
- [] Potrafię powiedzieć, po co jest `_delta_log`
- [] Potrafię wyjaśnić time travel bez magic phrases
- [] Potrafię nazwać 3 scenariusze, gdzie Delta jest lepszy niż Parquet
- [] Potrafię rozróżnić Delta i Iceberg
- [] Potrafię zapisać prostą tabelę Delta i odczytać poprzednią wersję

Jeśli choć 4 z 6 punktów są jasne, to lekcja jest zrozumiała na poziomie praktycznym.

 Pytanie 1

Dlaczego plain Parquet nie wystarcza w produkcji?

 Pytanie 2

Czym różni się Delta od zwykłego folderu danych?

 Pytanie 3

Kiedy użyjesz ``versionAsOf`` albo ``history()``?

 Pytanie 4

Dlaczego ``mergeSchema`` ma sens przy ewolucji danych?

Delta Lake nie jest „nowa ekstencja pliku”.

To jest nowy model pracy z danymi:

- tabele mają stan,
- mają historię,
- mają walidację i rollback,
- dają bezpieczniejszy pipeline.

To jest fundament, który pozwala przejść od "pliki na dysku" do "produkcyjnej platformy danych".

Homework i interview

Homework: build Delta pipeline + time travel + schema evolution

Interview: explain why raw files are not enough in production

Najważniejsze pytanie:

Czy Twoje dane są tylko plikami, czy są też wersjonowaną tabelą?

